Lebanese American University
Department of Computer Science and Mathematics
CSC 380 - Theory of Computation
Spring 2025

Instructor: Faisal N. Abu-Khzam

1 Formal Languages: a course introduction

In most of your courses you learned what computers can be used for and how to solve computational problems. In this course you learn more about what is solvable and what is unsolvable by conventional computers.

We begin with formal languages. These are languages that can be defined mathematically in the same way we define sets.

Definition 1. *An* **Alphabet** *is a finite non-empty set of symbols.*

Throughout this course we use the symbol Σ to denote an alphabet. Examples of alphabets include the binary alphabet $\Sigma = \{0, 1\}$, the English Alphabet (lower and/or upper-case letters), the ASCII characters, etc...

Definition 2. *A* **String** *is a finite sequence of symbols from a specific alphabet.*

Definition 3. *The length of a string is the number of symbols forming it (i.e., number of elements of the finite sequence). The string of length zero is the* **empty string**, *denoted by ϵ in this text.*

A symbol that is an element of an alphabet Σ can also be treated as a string of length one, depending on the context in which it is used. As such Σ can be identified with the set of strings of length one over Σ. To denote strings of a certain specific length over Σ we define the power of an alphabet as follows.

Definition 4. *Let Σ be an alphabet. We denote by Σ^k the set of strings of length k over Σ.*

For example, if $\Sigma = \{0, 1\}$, then $\Sigma^0 = \{\epsilon\}$, $\Sigma^1 = \{0, 1\}$, $\Sigma^2 = \{00, 01, 10, 11\}$, etc... Note that Σ^1 is not the same as Σ. The former is a set of strings of length one while the latter is a set of symbols.

The set of all strings over an alphabet Σ is denoted by Σ^*, which can formally be defined as:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cdots = \bigcup_{i=0}^{\infty} \Sigma^i$$

If we exclude the empty string we obtain the set:

$$\Sigma^+ = \Sigma^* - \{\epsilon\} = \bigcup_{i=1}^{\infty} \Sigma^i$$

Definition 5. *Let x and y be two strings. We denote by xy the concatenation of x and y. The concatenation of a string x with itself n times can be denoted by x^n. As such, $x^0 = \epsilon$ since it corresponds to zero occurrences (or copies) of x. This must not be confused with algebraic rules of exponentiation.*

Definition 6. *A* **language** *over an alphabet Σ is a subset of Σ^*, or just a set of strings formed by symbols that are elements of Σ.*

Here are a few examples of languages over $\Sigma = \{0, 1\}$

- The set of strings of even length: $\{\epsilon, 00, 01, 10, 11, 0000, 0001, 0010, 0011, \cdots\} = \{w \in \{0,1\}^* : |w| \text{ is even}\}$.
- The set of strings where any zero must precede any occurrence of a 1 $= \{0^i 1^j : i, j \geq 0\} = \{\epsilon, 0, 1, 00, 01, 11, 000, 001 \cdots\}$.
- Σ^*
- The empty set ϕ, being a subset of Σ^*.
- $\{0^p : p \text{ is prime}\} = \{00, 000, 0^5, 0^7, \cdots\}$
- $\{0^n 1^n : n \geq 0\} = \{\epsilon, 01, 0011, \cdots\}$
- $\{(01)^n : n \geq 0\} = \{\epsilon, 01, 0101, \cdots\}$

Languages and Problems

Recall that a set E is countable if and only if there is a one to one correspondence between E and a subset of the set $\mathcal{N}$ of the natural numbers. The set $\mathcal{N}$ itself is thus countable, while the power set of $\mathcal{N}$ is uncountable. The same is true for the power set of any infinite set.
Let $\Sigma = \{0, 1\}$. Then $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000 \cdots\}$ is in a one-to-one correspondence with $\mathcal{N} - \{0\}$. To see this, note that each binary string can be mapped to a positive integer by adding 1 to its left (via concatenation) and treating it as a binary integer (integer in base two). Therefore Σ^* is countable while its power set (set of all languages over Σ) is uncountable.
A programming language can also be seen as a set of strings over an alphabet. For simplicity, assume the said alphabet is $\Sigma = \{0, 1\}$. As such, the language can be seen as the set of syntactically correct computer programs. So **the set of all programs can be seen as a subset of Σ^*.**
A decision problem is one whose required answer/output for any given/input is either Yes or No. The input can be assumed to be a string.

An input string for a problem X that causes the answer to be Yes is referred to as a Yes-instance of X. Otherwise the string is a No-instance. The set of all Yes-instances of X is a language L_X, being a set of strings.
We therefore assume the set of languages is equivalent to (or in 1-1 correspondence with) the set of decision problems. This stems from the fact that every decision problem gives rise to a language, as we just saw, and any language L can be seen as the set of Yes-instance of a problem, namely the problem that takes a string as input and asks whether the string belongs to L.
Therefore **the set of decision problems is equivalent to the power set of Σ^* (i.e., the set of all subsets of Σ^*).**
It follows from the above discussion that the set of all decision problems is uncountable while the set of all (computer) solutions is countable. This implies that, unfortunately, there are unsolvable decision problems.

2 Deterministic Finite Automata

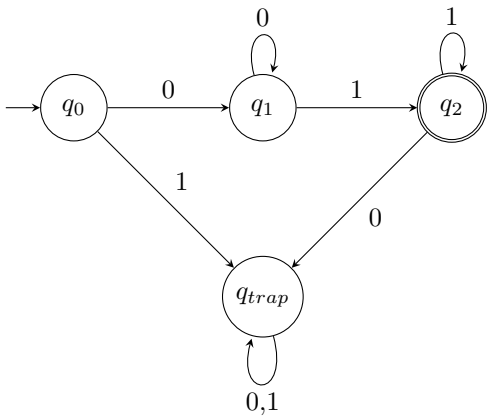
From this point on we deal with the following type of problems.

Given: a description of a language L and a string x
Question: is $x \in L$?

A deterministic finite automaton (DFA) is a machine characterized by a quintuple $(Q, \Sigma, \delta, q_0, F)$ where:

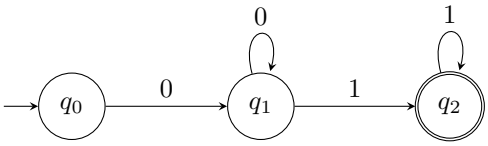
- Q is a finite set of states
- Σ is the alphabet (set of symbols the machine can process)
- $\delta : Q \times \Sigma \longrightarrow Q$ is the transition function
- q_0 is the initial state (special state at which the processing starts)
- $F \subseteq Q$ is a set of final (or accepting) states

Deterministic finite automata are used to solve the above typical problem (determine whether a given string is an element of a given language). A DFA is assumed to be in its starting state before reading (or processing) any symbol. It reads the input string by only moving right one symbol at a time until the end-of-input is reached. While we do not specify any delimiter, we assume the DFA stops after reading the last symbol of its input string. It is possible, however, that the input has no symbols, in which case we do not assume it to be empty: it consists of the empty string ϵ. After reading the last symbol of its input, the DFA accepts the string if it is in a final state, otherwise it rejects. As an example, the following DFA recognizes the language $L = \{0^i 1^j : i, j > 0\}$.

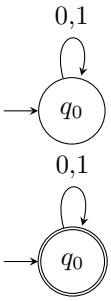


Note that δ is assumed to be a total function, which means that we must have a transition for every $(state, symbol)$ pair. While a DFA has no memory, each state can correspond to processing some "string type" or somehow knows a simple-to-guess property of the string processed so far. For example, when the

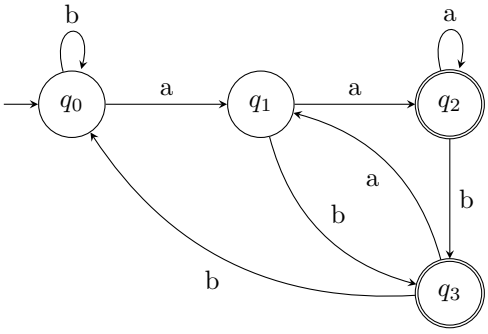
machine is in state q_1 in the above DFA we know the string processed so far consists of 0s only.
In some cases, reading a particular symbol must result in rejecting the input string, even if the DFA has more symbols to read. In this case the DFA goes into what we call a *trap state* since the only condition to stop is to reach the end of input, as explained above. In the above example, q_{trap} is a trap state since it captures the two forbidden cases where a string either starts with a 1 or contains a zero after a 1.
Having a trap state in some DFA could be confusing especially if the number of transitions is high. An elegant way to avoid this is to omit the trap state and assume that "missing arrows/transitions lead to a trap state." For example, the below DFA is assumed to be equivalent to the above one.



The empty set and the set of all strings can both be recognized by DFA. Below are the two DFA for $\emptyset$ and Σ^* respectively, assuming $\Sigma = \{0, 1\}$.



An example of a more sophisticated DFA is given below for the langugae $\{w \in \{a, b\}^* : \text{second symbol from the right in } w \text{ is an } a\}$.



The Language of a DFA

Recall that a DFA reads the input string one symbol at a time, and the transition function gives the state reached after reading a single symbol. It is often important to determine the state reached after processing a string from a given state. For this purpose we extend (or augment) δ into the following:

$$\hat{\delta} : Q \times \Sigma^* \longrightarrow Q$$

defined by:

$$\hat{\delta}(q, \epsilon) = q,$$
$$\hat{\delta}(q, ax) = \hat{\delta}(\delta(q, a), x), \ if \ x \ is \ a \ string.$$

Definition 7. *For a DFA $M = (Q, \Sigma, \delta, q_0, F)$, we define the language of M by $L(M) = \{w \in \Sigma^* : \hat{\delta}(q_0, w) \in F\}$.*

A language that is not recognized by DFA

Consider the language $\{0^n 1^n : n \geq 0\}$. We prove that L cannot be recognized by a DFA by contradiction, as follows.
Assume there is a DFA M for L. Then M has a finite (but unknown) number of states, say k. Consider the string $w = 0^k 1^k$. Since $w \in L$, it must be accepted by M. We think of processing w by M as a traversal of the corresponding directed graph.
Since the number of symbols in w is the same as the number of transitions made by M upon processing w, and the number of states in M is less that $|w|$, some state q_i must be revisited while processing w. It follows that (the digraph of) M has a cycle C labeled with symbols 0s and/or 1s.
Note that traversing C twice leads to the same final state reached upon processing w. If the arcs forming C have both 0 and 1 labels, then traversing C twice leads to accepting a string that has a 0 after a 1, which is impossible since L does not contain such a string. On the other hand, if C consists of 0s only (or 1s only) then traversing C twice (or more than the number of times traversed while processing w) leads to reading more 0s than 1s (or more 1s than 0s, respectively) before reaching the same final state and accepting a string that is not in L, which is the desired contradiction.

Homework I

Give the transition diagrams of DFA's accepting the following languages over the alphabet $\{0, 1\}$.

1. The set of all strings containing at most three zeros.
2. The set of all strings containing at least three zeros.
3. The set of strings that either start with a 1 or end with a 1.

4. The set of all strings that do not start and end with the same symbol.

5. $\{0^i 1^j : i + j > 1\}$

6. $\{0^i 1^j : i + j$ is odd$\}$

7. The set of all strings such that each block of four consecutive symbols contains at least two 0's.

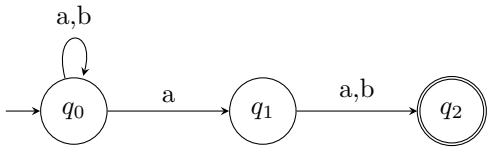
8. The set of binary strings that represent non-negative integers (in binary) that are multiples of 5 ($\{0, 101, 1010, 1111, \ldots\}$).

3 Non-deterministic Finite Automata

A non-deterministic finite automaton (NFA) is a machine characterized by a quintuple $(Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states
- Σ is the alphabet (set of symbols the machine can process)
- $\delta : Q \times \Sigma \longrightarrow 2^Q$ is the transition function
- q_0 is the initial state (special state at which the processing starts)
- $F \subseteq Q$ is a set of final (or accepting) states

As in the case of a DFA, a NFA is assumed to be in its starting state before reading (or processing) any symbol. It reads the input string by moving right one symbol at a time until the end-of-input is reached. Unlike DFA, however, a NFA is allowed to try more than one choice upon processing a symbol. Its transition function maps a $(state, symbol)$ pair to a set of states (which could very well be a singleton). As an example, the following NFA recognizes the language $\{w \in \{a, b\}^* : \text{second symbol from the right in } w \text{ is an } a\}$, for which we had a DFA in the previous section.



Exercise 1. *Give a non-deterministic finite automaton for each of the following languages. Your answers must show the effective use of non-determinisim whenever possible.*

1. $\{w \in \{0,1\}^* : w \textit{ contains } 00 \textit{ as substring}\}$
2. $\{a^i b^j : i + j \textit{ is even}\}$

Language of a NFA

The transition function of a NFA gives the set of states reached after reading a single symbol. As with DFA, it is often important to determine the state reached after processing a string from a given state. We can also extend (or augment) the transition function δ into the following:

$$\hat{\delta} : Q \times \Sigma^* \longrightarrow 2^Q$$

defined by:

$$\hat{\delta}(q, \epsilon) = \{q\},$$
$$\hat{\delta}(q, ax) = \bigcup_{p \in \delta(q,a)} \hat{\delta}(p, x), \ if \ x \ is \ a \ string.$$

Definition 8. *For a NFA $M = (Q, \Sigma, \delta, q_0, F)$, we define the language of M by $L(M) = \{w \in \Sigma^* : \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$.*

The Subset Construction Algorithm

Covered during interactive online session. If you were absent, the algorithm is well (and fully) described in sections 2.3.5 and 2.3.6 of the book.

4 Regular Expressions and Languages

So far we saw how a DFA or NFA is used to provide a finite description of a language by providing a mechanism for deciding whether a given string belongs to the language. And we saw this is not always possible. We now provide another type of a *finite description* of a language, namely via the notion of a regular expression.
A regular expression is used to describe how strings of a certain language are obtained or (informally) how they "look like." Regular expressions are defined inductively as follows.

- $\emptyset$ is a regular expression for the language $\emptyset$: $L(\emptyset) = \emptyset$.
- ϵ is a regular expression for the (non-empty) language that contains the empty string only: $L(\epsilon) = \{\epsilon\}$.
- A symbol a (of the alphabet in question) is a regular expression for the language $\{a\}$: $L(a) = \{a\}$
- If E and F are regular expressions then $E + F$ is a regular expression for $L(E) \cup L(F)$.
- If E and F are regular expressions then EF is a regular expression for $L(E)L(F)$ (the concatenation of the two languages).
- If E is a regular expression then E^* is a regular expression for $L(E)^*$.
- If E is a regular expression then (E) is another regular expression for $L(E)$.

Example 1. *Below are a few regular expressions with the corresponding languages.*

1. $(0+1)^*$ *is a regular expression for* $\{0,1\}^*$ *since* $L((0+1)^*) = (L(0+1))^* = (L(0) \cup L(1))^* = (\{0\} \cup \{1\})^* = \{0,1\}^*$.

2. $(0+1)^*1$ *is a regular expression for the language of all binary strings that end with a 1.*

From a DFA to a regular expression

Not required.

From a regular expression to a DFA

Section 3.2.3 in the book. Construction of an ϵ-NFA from a regular expression is required (but there won't be "direct" exam questions on this topic).

Algebraic laws for regular expressions

Section 3.4 of the book. No direct (exam) questions from this section, but you must read it.

5 The Pumping Lemma for regular languages

Lemma 1 (The Pumping Lemma for regular languages.). *If L is a regular language, Then there is a constant n such that: if $w \in L$ and $|w| \geq n$ then $w = xyz$ such that:*

1. $|y| > 0$,
2. $|xy| \leq n$, *and*
3. $xy^k z \in L \quad \forall k \geq 0$.

We have already mentioned the fact that the language $\{a^n b^n : \ n \geq 0\}$ cannot be recognized by a DFA, and we implicitly used the above. Here is how the lemma will be used for a formal proof:

Proof. Assume $L = \{a^n b^n : \ n \geq 0\}$ is regular and let n be the constant of the PL. Choose: $w = a^n b^n$, then $w \in L$ and $|w| \geq n$. Therefore, by the PL, $w = xyz$ such that:

(i) $|y| > 0$

(ii) $|xy| <= n$

(iii) $xy^k z$ is in L for all $k \geq 0$

Observe: by (i) and (ii) $y = a^t$ such that $t > 0$.
Choose $k = 2$, get $xy^k z = xyyz = a^{n+t} b^n \notin L$. This is a contradiction with assertion (iii) of the PL.
Therefore our assumption is false, and $\{a^n b^n : \ n \geq 0\}$ is not regular.

□

Example 2. *The language $L = \{0^i 1^j : i >= j\}$ is not regular.*

Proof. Assume L is regular and let n be the constant of the PL. Choose: $w = 0^n 1^n$, then $w \in L$ and $|w| \geq n$. Therefore, by the PL, $w = xyz$ such that:

(i) $|y| > 0$

(ii) $|xy| <= n$

(iii) $xy^k z$ is in L for all $k \geq 0$

Observe: by (i) and (ii) $y = 0^t$ such that $t > 0$.
Choose $k = 0$, get $xy^k z = xz = 0^{n-t} 1^n \notin L$.
This is a contradiction with assertion (iii) of the PL.
Therefore our assumption is false, and L is not regular.

□

Example 3. *The language $L = \{0^i 1^j : i + j$ is a perfect square$\}$ is not regular.*

Proof. Assume L is regular and let n be the constant of the PL. Choose: $w = 0^{n^2}$, then $w \in L$ and $|w| \geq n$. Therefore, by the PL, $w = xyz$ such that:

(i) $|y| > 0$

(ii) $|xy| <= n$

(iii) xy^kz is in L for all $k \geq 0$

Observe: by (i) and (ii) $y = 0^t$ such that $0 < t \leq n$.
Choose $k = 2$, get $w' = xy^kz = xyyz = 0^{n^2+t}$.

$$0 < t \leq n$$
$$n^2 < n^2 + t \leq n^2 + n$$
$$n^2 < |w'| < n^2 + 2n + 1 = (n+1)^2$$

Thus $|w'|$ is not a perfect square (since $n^2 < |w'| < (n+1)^2$).
This is a contradiction with assertion (iii) of the PL.
Therefore our assumption is false, and L is not regular.

□

Practice problems. *Which of the following languages is/are regular? Prove your answers.*

1. $\{0^p : p \text{ is prime}\}$

   Solution. *Claim L is not regular.*

   Proof. Assume L is regular and let n be the constant of the PL. Choose $w = 0^p$, where p is a prime number that is $\geq n$. Then $w \in L$ and $|w| \geq n$. By the PL, $w = xyz$ such that:

   (i) $|y| > 0$

   (ii) $|xy| <= n$

   (iii) xy^kz is in L for all $k \geq 0$

   By (i) and (ii), $y = 0^t, 0 < t \leq n \leq p$.
   Choose $k = p + 1$ get $w' = xy^kz = 0^{p+(k-1)t} = 0^{p+pt} = 0^{p(1+t)} \notin L$ since $p(1+t)$ is composite $(p > 1, 1 + t > 1)$

   □

2. $\{w \in \{a, b\}^* : w \text{ consists of an equal number of } a's \text{ and } b's\}$

   Solution. *Claim L is not regular.*

   Proof. Assume L is regular and let n be the constant of the PL. Choose $w = a^nb^n$, then $w \in L$ and $|w| \geq n$. By the PL, $w = xyz$ such that:

(i) $|y| > 0$

(ii) $|xy| <= n$

(iii) $xy^k z$ is in L for all $k \geq 0$

Observe: by (i) and (ii) $y = a^t$ such that $t > 0$.

Choose $k = 2$, get $w' = xy^2z = xyyz = a^{n+t}b^n \notin L$. This is a contradiction. Therefore assumption is false and L is not regular.

□

3. $\{a^n b^m : n > 2m\}$

4. $L = \{0^n : \ n \ is \ a \ perfect \ square\}$.

5. $\{(ab)^n : n \ is \ a \ multiple \ of \ 3\}$

6. $\{a^i b^j c^k : i, j, k \geq 0\}$

6 Closure properties of regular languages

The set of regular languages is closed under:

- Union

  Reason: If L and L' are regular languages with regular expressions E and F, then $L \cup L'$ is regular, being the language of the regular expression $E + F$.

- Concatenation

  Reason: If L and L' are regular languages with regular expressions E and F, then LL' is regular, being the language of the regular expression EF.

- Complement

  Reason: easy to construct a DFA for $\overline{L}$ from a complete DFA for L: just swap accepting and non-accepting states.

- Intersection

  Reason: L and L' are regular $\implies$ $\overline{L}$ and $\overline{L'}$ are regular $\implies$ $\overline{(\overline{L} \cup \overline{L'})}$ is regular $\implies$ $L \cap L'$ is regular (by De Morgan's Law).

  In fact, if we have DFAs for L and L' we can construct a DFA for $L \cap L'$ (as discussed in class. We refer to this construction by the **Intersection Algorithm**

- Reversal

  Reason: If we have a DFA M for L we construct and ϵ-NFA M' for L^R as follows: reverse all arcs; introduce one initial state for M' and add ϵ transitions from it to the final states (of M); making all final states of M non-final in M' then making the initial state final of M final in M'.

Exercise 2. *True or False:*

1. *If L is regular and $L' \subset L$ then L' is regular.*
2. *If L is non-regular then $\overline{L}$ is non-regular.*
3. *The Non-regular languages are closed under union.*
4. *The Non-regular languages are closed under intersection.*
5. *The infinite union of regular languages is necessarily regular.*
6. *The set of regular languages is closed under infinite intersection.*

7 Decision Problems on DFAs

7.1 Testing emptiness of the language of a DFA

Let M be a given DFA. Testing whether $L(M)$ is non-empty can be done via breadth-first search by checking whether a final state can be reached from the initial state.

Therefore, there exists a decision algorithm that takes a DFA M and decides whether $L(M)$ is empty. We refer to this algorithm by **isEmpty** in the sequel.

7.2 Testing whether the language of a DFA is infinite

Let M be a given DFA. To check whether $L(M)$ is infinite we do the following:

First, we make sure M does not have any redundant state, simply by deleting any such state.
Note that a state q is redundant if one of the two conditions hold: (i) q is not reachable from the initial state or (ii) q is not final and there is no path q to a final state.

Second, we use depth-first search to detect whether the transition graph (or diagram) of M contains a cycle. Of so, then $L(M)$ is obviously infinite. Otherwise it is finite.

Therefore, there exists a decision algorithm that takes a DFA M and decides whether $L(M)$ is infinite. We refer to this algorithm by **isInfinite** (or **isFinite**) in the sequel.

Exercise 3. *Is the following problem decidable? Prove your answer.*

Given: a DFA M
Question: Does M accept a string of length > 10?

Solution. We prove the problem is decidable. First, let $isInfinite$ be the decision algorithm (described in class) which takes a DFA and determines whether its language is infinite or not. Moreover, for a DFA M and a string w, let $Accept(M, w)$ be the algorithm that decides whether M accepts w.
Here is a decision algorithm for the problem.

```
Input: DFA M
If isInfinite(M)
   Return YES
For i = 11 to |Q(M)| − 1 do
   For each string w of length i in Σ* do
      If Accepts(M, w)
         Return YES
Return NO
```

Here $Q(M)$ is the set of states of M. The main idea is: if $L(M)$ is infinite then definitely M accepts (infinitely many) strings of length > 10. On the

other hand, if $L(M)$ is finite then the longest string accepted by M has length $< |Q(M)|$ (less than the number of states in M). So in this latter case we try all the (finitely many) strings of length > 10 whose length is less than $|Q(M)|$.

Exercise 4. *Is the following problem decidable? Prove your answer.*

Given: DFAs M_1, M_2
Question: Is $L(M_1) \subset L(M_2)$*?*

Solution. We prove the problem is decidable. First, let $Intersect$ be the algorithm (described in class) that takes two DFAs as input and returns a DFA for the intersection of their languages and let $Complement$ be the algorithm (also described in class) that takes a DFA as input and returns the DFA for the complement of its language. Moreover, let $isEmpty$ be the algorithm that takes a DFA as input and decides whether its language is empty.
Now observe that $L_1 \subset L_2$ is equivalent to $L_1 \cap \overline{L_2} = \emptyset$. The above problem can be decided via the following algorithm.

Input: DFAs M_1, M_2
$M_2' \leftarrow Complement(M_2)$
$M_3 \leftarrow Intersect(M_1, M_2')$
If $isEmpty(M_3)$
  Return YES
Return NO

Remark 1. *Note that we can now decide whether two DFAs* M *and* M' *are equivalent (i.e., they recognize the same language), simply by running the above algorithm to check whether* M *is a subset of* M' *and vice versa.*

Exercise 5. *Is the following problem decidable? Prove your answer.*

Given: a DFA M
Question: Does M *accept a string of odd length?*

Solution. Let M_{odd} be a DFA that decides the language $\{w \in \Sigma^* : |w| \text{ is odd}\}$. We provide the following decision algorithm.

Input: DFA M
$M_2 \leftarrow Intersect(M, M_{odd})$
If $isEmpty(M_2)$
  Return NO
Return YES

Exercise 6. *[Homework] Give a decision algorithm that takes a DFA* M *as input and decides whether:*

1. M *does not accept more than 10 strings.*
2. M *accepts at least one non-empty string of length* > 100 *that starts and ends with the same symbol.*

3. *None of the strings accepted by M contains two consecutive zeros.*

4. *At least one string that is accepted by M starts with a 1.*

5. *M accepts at most two strings of length ≥ 100.*

8 Context Free Grammars

A context free grammar (or CFG) is a set of rules that determine how strings of a certain formal language can be produced (or how they are formed). A CFG is formally described as a quadruple (V, T, S, P) where:

- V is a set of variable symbols. These are symbols that can be replaced by strings from $(V \cup T)^*$ in the process of producing (or deriving) strings.
- T is a set of terminals, which are nothing but symbols from a certain given alphabet.
- $S \in V$ is the start symbol.
- P is the set of production rules, which determine how a variable symbol is replaced while deriving strings.

Example 4. *A CFG for the language* $\{a^n b^n : n \geq 0\}$ *is described by the following production rules (the set* P*)*

$S \to aSb$
$S \to \epsilon$

In this case, $V = \{S\}$*,* $T = \{a, b\}$ *and* $P = \{S \to aSb, S \to \epsilon\}$*. Note that we can combine production rules for the same variable (on the left) using* $|$ *(the or sign) as follows:* $S \to aSb|\epsilon$*.*

Example 5. *A CFG for the set of all palindromes over* $\Sigma = \{0, 1\}$ *is given by the following production rules*

$S \to 0S0|1S1|0|1|\epsilon$

Example 6. *A CFG for* $L(0^*)$ *is:*

$S \to 0S|\epsilon$

Example 7. *A CFG for the language* $\{a^n b^m : n > m\}$ *is described by the following production rules.*

$S \to AE$
$A \to aA|a$
$E \to aEb|\epsilon$

Note that A is responsible for producing one of more a's while E is responsible for producing an equal number of a's and b's.

8.1 String derivation and Context-Free Languages

Given a CFG $G = (V, T, S, P)$, we say that a string w can be *derived* from G if starting from S, a sequence of applications of production rules results in w. For example, the string *aaaaabb* can be derived from S in the CFG of Example 5.2.4 as follows:

$$S \Rightarrow AE \Rightarrow aAE \Rightarrow aaAE \Rightarrow aaaE \Rightarrow aaaaEb \Rightarrow aaaaaEbb \Rightarrow aaaaabb$$

The above derivation is a *leftmost derivation* in the sense that a leftmost variable is always *expanded* first at any step during the derivation (in other words, the variable symbol is replaced with the right hand side of a production rule). The string can also be derived via a *rightmost* derivation:

$$S \Rightarrow AE \Rightarrow AaEb \Rightarrow AaaEbb \Rightarrow Aaabb \Rightarrow aAaabb \Rightarrow aaAaabb \Rightarrow aaaaabb$$

Sometimes we wish to skip intermediate steps during the derivation of a string. For example, in the above derivation we may write:

$$S \stackrel{*}{\Longrightarrow} aaaE \stackrel{*}{\Longrightarrow} aaaaabb.$$

8.2 Reading material

Strings can also be derived using *parse trees*. See book for details. Also read about ambiguity of a CFG. This material is not required for the midterm exam.

8.3 Language of a grammar

Given a context-free grammar $G = (V, T, S, P)$, we define the language of G as follows:

$$L(G) = \{w \in T^* : S \stackrel{*}{\Longrightarrow} w\}$$

The language of a context-free grammar is said to be a **context-free language** (CFL for short.)

8.4 An example of a Non-CFL

The following language is not a CFL: $\{a^n b^n c^n : n \geq 0\}$

The proof of this assertion is skipped since it requires the Pumping Lemma for CFLs, which is not covered in this course. Instead, we shall use the stated fact as a theorem in the sequel.

8.5 Closure Properties of Context-Free Languages

8.5.1 CFLs are closed under union

Let $L_1 = L(G_1)$ and $L_2 = L(G_2)$ where $G_1 = (V_1, T_1, S_1, P_1)$ and $G_2 = (V_2, T_2, S_2, P_2)$. Further, assume $V_1 \cap V_2 = \emptyset$ (otherwise relabel the variables to make them distinct). Then $L_1 \cup L_2$ has the following CFG:

$(V_1 \cup V_2, T_1 \cup T_2, S, P)$ where $P = P_1 \cup P_2 \cup \{S \to S_1 | S_2\}$.

8.5.2 CFLs are closed under concatenation

Let $L_1 = L(G_1)$ and $L_2 = L(G_2)$ where $G_1 = (V_1, T_1, S_1, P_1)$ and $G_2 = (V_2, T_2, S_2, P_2)$. Again, we may assume $V_1 \cap V_2 = \emptyset$. Then $L_1 L_2$ has the following CFG:

$(V_1 \cup V_2, T_1 \cup T_2, S, P)$ where $P = P_1 \cup P_2 \cup \{S \to S_1 S_2\}$.

8.5.3 Every regular language is context-free

Just note that any regular expression can be turned into a context-free grammar (see book for details).

8.5.4 CFLs are not closed under intersection

We give an example of two CFLS whose intersection is a non-CFL.

Let $L_1 = \{a^n b^n c^i : n, i \geq 0\}$ and $L_2 = \{a^j b^m c^m : j, m \geq 0\}$.
Both L_1 and L_2 are context-free (exercise: given a CFG for each), while $L_1 \cap L_2 = \{a^k b^k c^k : k \geq 0\}$, which is not a CFL.

Of course, there are CFLs whose union is a CFL. To see this note that every regular language is a CFL and the union of any two regular languages is regular (thus CF).

Exercise 7. *Are CFLs closed under complement? Prove your answer.*

8.5.5 The intersection of a CFL and a regular language.

Theorem 1. *If L is context-free and L' is regular, then $L \cap L'$ is context-free.*

The proof is omitted. Of course, the statement is obvious when L is regular, since the intersection of two regular languages is regular (thus context-free).

Example 8. *Let $L = \{w \in \{a, b, c\}^* : w$ has equal number of a's, b's and c'c$\}$. We show that L is not a CFL as follows:*
Assume L is context-free and let $L' = L(a^* b^* c^*)$, then L' is trivially regular. By the above theorem, $\{a^n b^n c^n : n \geq 0\} = L \cap L'$ is context-free, which is not the case $\to\leftarrow$, a contradiction.

Exercise 8. *Give a context-free grammar for each of the following languages.*

1. $\{a^i b^j : i, j \geq 0\}$
2. $\{a^i b^j : i \neq j\}$
3. $\{a^i b^j : j = 2i\}$
4. $\{a^i b^j : i + j$ *is odd*$\}$
5. $\{a^i b^j c^k : j = i + k\}$
6. $\{a^i b^j c^k d^t : i + j = k + t\}$

Exercise 9. *Prove or disprove:*

1. *If L is regular and L' is non-regular then $L \cup L'$ must be non-regular.*

2. $\{a^n b^n c^{n+2} : n > 0\}$ *is not a context-free language.*

3. *The complement of a non-context free language is necessarily non-context free.*

4. *If two languages* L *and* L' *are such that:* L *is context-free and* L' *is regular, then* $L - L'$ *is context-free.*

5. *If two languages* L *and* L' *are such that:* L' *is regular,* $L \cup L'$ *is regular and* $L \cap L'$ *is regular, then* L *is regular.*

9 Push-down Automata

Covered briefly in class. Not required, but optional for proving that a language is context-free.

8. Turing Machines

A Turing machine M is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where:

- Q is the set of states;
- Σ: the input alphabet;
- Γ: tape alphabet;
- δ: transition function;
- q_0: the initial state;
- B: the blank symbol;
- F: set of final states.

A Turing machine consists mainly of a finite control (the states), a two-way infinite tape (can be seen as a two-way infinite array), and a head that is pointing to a cell of the tape. The input is surrounded by blank (all cells to the left and to the right contain the blank symbol B). At the beginning of any computation, the head points to the first input cell (or first symbol of input string). If the head points initially to a blank symbol then the input string is ϵ.

A Turing machine has the ability to:

- move right or left while traversing input;
- overwrite the content of a tape cell;
- accept without having to read all its input,

Example 9. *A Turing machine for the language* $\{a^n b^n : n \geq 0\}$

$\delta(q_0, B) = (q_4, B, R)$
$\delta(q_0, a) = (q_1, X, R)$
$\delta(q_0, Y) = (q_3, Y, R)$
$\delta(q_1, a) = (q_1, a, R)$
$\delta(q_1, Y) = (q_1, Y, R)$
$\delta(q_1, b) = (q_2, Y, L)$
$\delta(q_2, Y) = (q_2, Y, L)$
$\delta(q_2, a) = (q_2, a, L)$
$\delta(q_2, X) = (q_0, X, R)$
$\delta(q_3, Y) = (q_3, Y, R)$
$\delta(q_3, B) = (q_4, B, R)$

In tabular form:

State	a	b	X	Y	B
$\rightarrow q_0$	(q_1, X, R)	-	-	(q_3, Y, R)	(q_4, B, R)
q_1	(q_1, a, R)	(q_2, Y, L)	-	(q_1, Y, R)	-
q_2	(q_2, a, L)	-	(q_0, X, R)	(q_2, Y, L)	-
q_3	-	-	-	(q_3, Y, R)	(q_4, B, R)
$* \ q_4$	-	-	-	-	-

9.1 Exercises.

Give the transition table of a Turing machine for each of the following languages.

1. $L(0^*1^*)$

State	0	1	B
$\rightarrow q_0$	$(q_0, 0, R)$	$(q_1, 1, R)$	(q_2, B, R)
q_1	-	$(q_1, 1, R)$	(q_2, B, R)
$* \ q_2$	-	-	-

2. $\{0^n : n \text{ is odd}\}$

State	0	B
$\rightarrow q_0$	$(q_1, 0, R)$	-
q_1	$(q_0, 0, R)$	(q_2, B, R)
$* \ q_2$	-	-

3. $\{a^n b^m : n \geq m\}$
4. $\{ww^R : w \in \{0,1\}^*\}$
5. $\{a^n b^n c^n : n \geq 0\}$

9.2 Instantaneous Descriptions

An instantaneous description (id) is like a snapshot of a current or particular status of the machine. It includes the current state; the current head position and the current tape content. For example, assume the input to the above TM is the string aabb. Then the initial id is q_0aabbB. A transition from one id to another is denoted by $\vdash$ (Greek letter *Heta*). For example:

$$
\begin{aligned}
q_0aaabbbB &\vdash Xq_1aabbbB \\
&\vdash Xaq_1abbbB \\
&\vdash Xaaq_1bbbB \\
&\vdash Xaq_2aYbbB \\
&\vdash Xq_2aaYbbB \\
&\vdash q_2XaaYbbB \\
&\vdash Xq_0aaYbbB
\end{aligned}
$$

$$\vdash XXq_1aYbbB$$
$$\ldots$$

We can also jump over transitions using $\overset{*}{\vdash}$:

$q_0aaabbbB \overset{*}{\vdash} XXq_1aYbbB$

9.3 Language of a Turing Machine.

A string w is accepted by a TM M if M **halts** while/upon processing w and the state of the machine (when it halts) is final. The language of a TM is simply the set of strings accepted by the machine:

$$L(M) = \{w : w \text{ is accepted by } M\}$$

A string w is rejected by a TM M if M **halts** (i.e., stops) while/upon processing w and the state of the machine is not final (i.e., not an accepting state).

9.4 Other Turing Machine Models

Reading material (see book).

9. Undecidable Problems and Languages

9.5 Recursively Enumerable Languages.

A formal language L is *recursively enumerable* (r.e.) if there exists a Turing machine M that:
(i) accepts all the strings of L, and
(ii) does not accept any string that is not in L

9.6 Recursive Languages.

A formal language L is *recursive* if there exists a Turing machine M that:
(i) accepts all the strings of L, and
(ii) rejects any string that is not in L

9.7 Closure Properties.

Theorem 2. *The set of recursive languages is closed under union.*

Theorem 3. *The set of recursive languages is closed under intersection.*

Theorem 4. *The complement of a recursive language is recursive.*

Theorem 5. *The set of r.e. languages is closed under union.*

Theorem 6. *The set of r.e. languages is closed under intersection.*

Most importantly, the set of r.e. languages is not closed under complement. In fact, we have a much stronger statement in this case: the complement of an r.e. language L is r.e. only when L is recursive.

Theorem 7. *If the complement of an r.e. language is r.e., then it is recursive.*

9.8 Turing Machine Encoding

All TMs considered hereafter assume as alphabet $\Sigma = \{0, 1\}$ and as tape alphabet $\Gamma = \{0, 1, B\}$. Moreover, Σ^* is ordered according to the inverse of following mapping:

$$f : \Sigma^* \longrightarrow \mathcal{N}$$
$$\text{Such that } f(w) = 1.w$$

In other words, f^{-1} orders the strings in Σ^* as follows:
$1 \rightarrow \epsilon$, so ϵ is w_1
$10 \rightarrow 0$ ($w_2 = 0$)
$11 \rightarrow 1$ ($w_3 = 1$)
$100 \rightarrow 00$ ($w_4 = 00$)
etc...

Example 10. *To find w_i we first represent i in binary then remove the leading (leftmost) 1. For example, to find w_{17}:*
17 is 10001 *in binary. Therefore* $w_{17} = 0001$.
On the other hand, if $w_i = 11001$ *then* $i = (111001)_2 = 57$.

Note that f is a one-to-one mapping from Σ^* to $\mathcal{N}$, which proves that Σ^* is countable. We now show the set of all Turing machines is also countable, by showing it is in one-to-one correspondence with a subset of $Sigma^*$. To do this, we encode a Turing machine by a string as follows:

- The symbol $0, 1$ and B are encoded with $0, 00$ and 000 respectively.
- The directions L and R are encoded with 0 and 00 respectively.
- Each state q_i is encoded by i consecutive 0s.
- The initial state is q_1 (there is no q_0... obviously).
- The machine has only one final state denoted by q_2 and encoded with 00. This is always possible with TMs.
- The symbol 1 will be used as separator while encoding a transition, and the string 11 will be used as to separate between two transitions.

Example 11. *A TM for the language $L(0^*1^*)$ is:*

$\delta(q_1, B) = (q_2, B, R)$	*010001001000100*
$\delta(q_1, 0) = (q_1, 0, R)$	*0101010100*
$\delta(q_1, 1) = (q_3, 1, R)$	*01001000100100*
$\delta(q_3, 1) = (q_3, 0, R)$	*000100100010100*
$\delta(q_3, B) = (q_2, B, L)$	*0001000100100010*

The corresponding encoding is:
0100010010001001101010101001101001000101001100010010001001001100010001001001000010

We denote a TM by M_i if its encoding is the binary string w_i. Of course, there are numbers i such that w_i is not a valid TM encoding $(1, 2, \cdots)$. However, we adopt the convention that: M_i is a TM that accepts nothing in the case where w_i is not a valid TM encoding. This applies to TMs with small encoding such as $M_1, M_2, \cdots$.

Problem 1. *What is the smallest number k such that $L(M_k) \neq \emptyset$.*

9.9 The non-R.E. Languages

Obviously, every r.e. language has a Turing machine that can be encoded and thus corresponds to a string in Σ^*. In fact, for each r.e. language L we can associate the smallest string that encodes a TM for L. As such, we can have a

one-to-one (but not onto) mapping from the set of r.e. languages to the set of binary strings. This proves that the set of r.e. languages is countable.

The set of all languages over $\Sigma = \{0, 1\}$ is uncountable since it is in one-to-one correspondence with the power set of $\mathcal{N}$. Therefore the set of languages that cannot be recognized by Turing machines is uncountably infinite, while those that can be recognized by Turing machines is countable.

9.10 A language that is not recursively enumerable

Define $L_d = \{w = w_i : w \notin L(M_i)\}$.

Theorem 8. *L_d is not recursively enumerable.*

Proof. Assume L_d is r.e. then there exists a TM for L_d whose encoding is a string w_k. In other words $L_d = L(M_k)$ for some k.
Since L_d is a well defined language, every string is either in L_d or not. Now consider the string w_k. If $w_k \in L_d$ then (according to L_d's definition) $w_k \notin L(M_k) \implies w_k \notin L_d$.

□

9.11 Some R.E languages

9.11.1 $\overline{L_d}$

$\overline{L_d} = \{w = w_i : w \in L(M_i)\}$

A TM M for $\overline{L_d}$ simply takes a string w as input, and simulates the corresponding TM: the TM M_i such that $w = w_i$. If M_i accepts w, then M accepts w. No need to think about the other option!

9.11.2 The Universal Language

Define: $L_u = \{< M, w >: w \in L(M)\}$. In other words, L_u contains strings of the form $w111w'$ where w' is a string and w is the encoding of a Turing machine that accepts w'.

The fact that L_u is r.e. is obvious. Same idea as in the proof of $\overline{L_d}$.

9.11.3 Non-emptiness property

Let $L_{ne} = \{< M >: L(M) \neq \emptyset\}$
The book shows L_{ne} is r.e. using a non-deterministic TM. There is a better proof using pair generators. While this is not required for the final exam, it will be discussed in class.

9.11.4 Is $\overline{L_d}$ recursive?

$\overline{L_d}$ is not recursive since otherwise its complement L_d would be recursive, but it's not even r.e.

9.11.5 Is L_u recursive?

L_u is not recursive because of the following "reduction" from $\overline{L_d}$.
Assume L_u is recursive, then there exists an algorithm A for L_u. We can use A to decide $\overline{L_d}$ as follows: Give a string $w = w_i$, generate the corresponding Turing machine M_i and pass $< M_i, w_i >$ to A. Then A's output is Yes if $w_i \in L(M_i)$ and No otherwise, thereby deciding whether $w_i \in \overline{L_d}$, which is impossible since $\overline{L_d}$ is not recursive.

9.12 The Halting problem

Read section 8.1. This section is not required for exams.

9.13 Rice's Theorem

We now consider languages of the form $L = \{< M >: L(M)$ satisfies property $P\}$. In other words, every element of L is the encoding of a Turing machine that satisfies a certain property P. For each such language L (and property P), we pose the question: is the language recursive? This is equivalent to asking: is the property P decidable? In other words, is there an algorithm (i.e., TM that halts on all input) that decided the property? Equivalently: is there a TM that halts on all input for L. Examples of languages of strings that encode TMs include:

(i) $\{< M >: L(M) = \phi\}$
(ii) $\{< M >: L(M)$ is regular$\}$
(iii) $\{< M >: L(M)$ is finite$\}$

Luckily, we have an easy tool for the above. Unfortunately, however: the tool is easy because almost all the answers to the above questions are negative.

Theorem 9 (Rice's Theorem). *Every non-trivial properly of the r.e languages is undecidable*

By non-trivial, the Rice's theorem means the following: neither all nor none.

Example 12. *Let $L_1 = \{< M >: L(M)$ is regular$\}$. Is L_1 recursive?*

Solution. First we note the following:

(i) ϕ is r.e. and regular, so the regularity property is not empty (it contains at least one language).

(ii) $\{0^n 1^n : n \geq 0\}$ is r.e. and non-regular, so the regularity property does not contain all the r.e. sets.

By (i) and (ii), the property of being regular is a non-trivial property of the r.e. languages. Therefore, by Rice's theorem the property is undecidable. Hence L_1 is non-recursive.

Example 13. *Let $L_2 = \{< M >: L(M)$ is infinite$\}$. Is L_2 recursive?*

Solution. First we note the following:

(i) Σ^* is r.e. and infinite, so the infiniteness property is not empty (it contains at least one language).

(ii) $\{0\}$ is r.e. and finite, so the infiniteness property does not contain all the r.e. sets.

By (i) and (ii), the property of being infinite is a non-trivial property of the r.e. languages. Therefore, by Rice's theorem the property is undecidable. Hence L_2 is non-recursive.

Example 14. *Let* $L_3 = \{< M >: L(M)$ *is r.e.*$\}$. *Is* L_3 *recursive?*

Solution. All the r.e. languages are r.e., so this property (of being r.e.) is trivial. Therefore, by Rice's Theorem, L_3 is recursive.

Example 15. *Let* $L_4 = \{< M >: L(M)$ *is recursive*$\}$. *Is* L_4 *recursive?*

Solution. First we note the following:

(i) Σ^* is r.e. and recursive, so the recursiveness property is not empty (it contains at least one language).

(ii) L_u is r.e. but not recursive, so the property does not contain all the r.e. sets.

By (i) and (ii), the property of being recursive is a non-trivial property of the r.e. languages. Therefore, by Rice's theorem the property is undecidable. Hence L_1 is non-recursive.

The above example used to confuse students because it reads: "the property of being recursive is non-recursive" or "the property of being decidable is undecidable" or "decidability is undecidable." You should read these equivalent sentences carefully and make sure you understand them.

9.14 Exercises

1. Which of the following languages is/are recursive?
   (a) $\{< M >: 0 \in L(M)\}$
   (b) $\{< M >: L(M)$ is context-free$\}$
   (c) $\{< M >: L(M) = \phi\}$
   (d) $\{< M >: L_d \subset L(M)\}$
   (e) $\{< M >: L(M)$ is not r.e.$\}$
2. Which of the following problems is/are decidable?
   (a) Given a TM M; does M accept a string that starts with a 0?

(b) Given a TM M; does M accept a string of length > 10?

(c) Given a TM M; is $L(M)$ empty?

3. Is the set of non-recursive languages closed under complement? Prove your answer.

4. Is the set of non r.e languages closed under complement? Justify.

5. Is the set of non r.e languages closed under union or intersection? Prove your answer.

6. Consider the language $L_{ne} = \{< M >: L(M) \neq \phi\}$ (defined in 9.7.3). Is L_{ne} recursive? What can you say about its complement? Prove your answer.

10. NP-Completeness

We now consider the decidable problems (or recursive languages) only. The main question at this stage is how fast can we decide a problem. If a problem L is decidable but any conventional computer (or TM) takes several thousand years to solve some instances of L, then we have a problem. As you know this course has to do mainly with "computability." A problem that we cannot solve within a certain period (i.e., a solution is not expected to be obtained when it is needed) might very well be classified as unsolvable.

A non-deterministic TM is simply a TM that has a set of possible moves per transition. This is the same concept covered with finite automata (DFA versus NFA). Since a TM with many options tends to be much faster than a deterministic one, we distinguish between problems that are solvable in polynomial-time by a DTM (deterministic TM) and those that are solvable in polynomial-time by a NDTM. We thus define the following classes of problems/languages.

Definition 9 (**The class** P)**.** *A problem belongs to the class P if it is decidable/solvable by a DTM that is guaranteed to halt after a number of steps that is a polynomial function of the input size.*

Definition 10 (**The class** NP)**.** *A problem belongs to the class NP if it is decidable/solvable by a NDTM that is guaranteed to halt after a number of steps that is a polynomial function of the input size.*

We now distinguish between the terms "problem" and "instance." An instance of a problem is a particular input to the problem. For example, an instance of the sorting problem is a array that we wish to sort.

For decision problems, we say an instance is either a *Yes-instance* or a *No-instance* depending on whether the answer to the problem is yes or no on the particular input represented by the instance.

Two instances of two different problems are said to be equivalent if they are either both Yes-instances or both No-instances. We now define the following:

Definition 11. *Let X and Y be decision problems. We say that X is poly-time reducible to Y (denoted $X \propto Y$) if there exists an algorithm A such that:*
(i) A runs in polynomial time
(ii) A transforms an arbitrary instance of X to an equivalent instance of Y.

Definition 12. *A problem X is said to be $\mathcal{NP}$-hard if for every problem Y we have: $Y \in NP \implies Y \propto X$.*

Definition 13. *A problem X is said to be $\mathcal{NP}$-complete if:*
(i) $X \in NP$, and
(ii) X is $\mathcal{NP}$-hard.

Remark 2. *Let Y be any $\mathcal{NP}$-hard problem. If a problem X is such that $Y \propto X$, then X is $\mathcal{NP}$-hard.*

9.15 Boolean Satisfiability

Boolean Satisfiability, or SAT is the first problem shown to be $\mathcal{NP}$-complete (see Cook-Levin Theorem in the book). The problem is defined as follows:

Given: a Boolean formula F
Question: Is F satisfiable? In other words: is there a truth assignment to the variable (in F) that renders F true?

Here is a simple example; let $F = (x \vee \overline{y}) \wedge (\overline{x} \vee y) \wedge (\overline{x} \vee \overline{y})$. Then F is satisfiable since we cna make it true by setting $x = y = false$.

In the above example, x and y are *truth variables*; $\overline{x}$ and x appearing in the formula are also called *literals*. They are two different (opposite) appearances of the same variable.

CNF-SAT is the same problem but when the input Boolean formula is given in conjunctive normal form (CNF): a conjunction of clauses, each of which is a disjunction. Moreover, the kSAT problem is also a variant of SAT where the Boolean formula is in CNF and each clause consists of exactly k elements. In the above example, F is a 2-CNF formula, so it is an instance of the $2SAT$ problem.

It is shown (in the book) that $SAT \propto 3SAT$. We therefore use $3SAT$ in the sequel to prove the NP-hardness/completeness of a few classical problems.

9.15.1 Independent Set is NP-Complete

The Independent set problem (IS) is defined as follows.

Given: A graph $G = (V, E)$ and an integer $k > 0$
Question: Is there a set $I \subset V$ of cardinality k or more such that no two elements of I are adjacent?

First, we prove that $IS \in NP$, as follows.

Given an instance (G, k) of the Independent Set problem, together with a solution I, we verify in polynomial-time that I is a solution using the following algorithm:

- if $|I| < k$
  return No
- for each pair $\{u, v\}$ of elements of I do
  If u and v are adjacent
  return No
- return Yes

To prove the NP-hardness of IS we use a reduction from $3SAT$ as follows.

Given an instance $F = (x_{11} \vee x_{12} \vee x_{13}) \wedge (x_{21} \vee x_{22} \vee x_{23}) \wedge (x_{31} \vee x_{32} \vee x_{33}) \wedge \cdots$ of $3SAT$ where each x_{ij} is a literal (i.e., it can be a positive or negative appearance of a certain variable). For example, it is possible that $x_{11} = v$ and $x_{32} = \overline{v}$ or $x_{11} = x_{32} = v$ or $x_{11} = v$ and $x_{32} = \overline{w}$ etc...
To transform the above $3SAT$ instance into an equivalent instance of IS we first construct a graph $G = (V, E)$ as follows:

- For each literal in F we have a corresponding vertex: $V = \{x_{ij} : x_{ij}$ is a literal in $F\}$

  Now we must make sure a solution I of the constructed IS instance corresponds to a truth assignment of the variables of F and vice versa. We then continue our construction having in mind that elements of the IS solution will be assigned *true* as truth value and they must make each clause evaluate to true so that the overall conjunction (i.e. F) evaluates to true. For this purpose we must make sure that no two opposite literals are assigned the same truth value (true, in this case) by forcing at least one of them to be excluded from the solution, as follows:

- Add an edge between any pair of vertices that correspond to opposite literals.

  We must also provide a solution size (the value k in the definition) is given in the IS instance. This is the number of literals assigned the value *true*. For this purpose, and since each literal appears in exactly one clause, we must seek an independent set whose size equals the number of clauses in F. To guarantee this, we must make sure exactly one literal per clause is assigned *true* (i.e., selected in I) so we add edges between any two vertices whose corresponding literals belong to the same clause.

- For each clause $(x_{i1} \vee x_{i2} \vee x_{i3})$, add the edges $\{x_{i1}, x_{i2}\}$, $\{x_{i1}, x_{i3}\}$ and $\{x_{i2}, x_{i3}\}$.

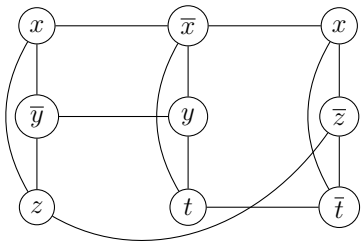


Figure 1: Reduction from $3SAT$ to IS

Figure 1 provides an illustration of how the formula $F = (x \vee \overline{y} \vee z) \wedge (\overline{x} \vee y \vee t) \wedge (x \vee \overline{z} \vee \overline{t})$ is transformed into an instance of 3-Independent Set.
Based on the above construction, we have now proved that F is a Yes-instance of $3SAT$ if and only if (G, k) is a Yes-instance of IS.

9.15.2 Independent Set $\propto$ Vertex Cover

The Vertex Cover proble (VC) is defined as follows.

Given: A graph $G = (V, E)$ and an integer $k > 0$
Question: Is there $S \subset V$ os cardinality k such that every edge of G has at least one endpoint in S?

We can easily show that $VC \in NP$ by verifying a solution as follows: for each $e = uv \in E$ check (in constant time) whether $u \in S$ or $v \in S$. (Exercise: write the pseudocode of the verification algorithm). To prove VC is NP-Complete we prove it NP-hard by reduction from IS. How?

9.15.3 Independent Set $\propto$ Clique

Easy! Reduce an arbitrary instance (G, k) of IS into an instance (G', k') of Clique by setting $G' = \overline{G}$ and $k' = k$. Note: the complement $\overline{G}$ of G is simply obtained (in poly-time) by swapping edges and non-edges.

9.15.4 Clique $\propto$ Subgraph Isomorphism

In the Subgraph Isomorphism problem we are given two graphs G_1 and G_2 and asked whether G_1 is isomorphic to a subgraph of G_2 (i.e., G_1 is a subgraph of G_2). This problem can be easily proved to be NP-hard by reduction from Clique:
Given an arbitrary instance (G, k) of Clique we produce (in poly-time) an instance (G_1, G_2) of SI as follows:
G_1 is a clique of size k, and $G_2 = G$.
Now we can easily prove that (G, k) is a Yes-instance of Clique if and only if (G_1, G_2) is a Yes-instance of SI (obvious!)

9.15.5 Vertex Cover $\propto$ Dominating Set

Not so easy: from an arbitrary instance (G, k) of Vertex Cover, we construct an instance (G', k') of DS as follows:
Start by initializing $G' = G$, then For each edge uv of G add a degree-two vertex, in G', that is adjacent to both u and v. Set $k = k'$. Now prove the two instances are equivalent. (Note any vertex cover in a graph is a dominating set).
Exercise: Complete the above proof.

9.15.6 Vertex Cover $\propto$ Set Cover

Easy!

9.15.7 Exercises

1. Knowing that 3SAT is NP-hard, show that 4SAT is NP-hard.
2. Knowing that 3-Coloring is NP-hard, show that 4-Coloring is NP-hard.

3. Prove the NP-hardness of each of the following problems.

   (a) Hitting Set.
   (b) Firehouse Problem (or distance d domination).
   (c) Induced Matching.
   (d) Deletion into Bounded Degree Graph.
   (e) Connected Dominating Set.